

Using Built-In Classes

Autoboxing Primitive Types

Autoboxing is the automatic conversion that the Java compiler makes between primitive types and their corresponding object wrapper classes. This includes converting an `int` to an `Integer` and a `double` to a `Double`.

```
// These are all examples of autoboxing
Double wrapper1 = 23.456;
Integer wrapper2 = 3;
Double wrapper3 = new Double(13.57);
Integer wrapper4 = new Integer(7);
Double wrapper5 = Double.valueOf(30.59);
Integer wrapper6 = Integer.valueOf(15);
```

Unboxing Wrapper Classes

Unboxing is the automatic conversion that the Java compiler makes from the wrapper class to the primitive type. This includes converting an `Integer` to an `int` and a `Double` to a `double`.

```
Integer IntegerWrapper = new Integer(7);

// Examples of an Integer getting unboxed to
an int
int primitiveInt = IntegerWrapper;
int primitiveInt2 =
IntegerWrapper.intValue();
```

Integer and Double Wrapper Classes

The `Integer` class and `Double` class are part of the `java.lang` package. They are used as a wrapper class to allow primitive `int` and `double` values to be used by methods that require Objects.

```
// Primitives
int a = 5;
double b = 2.5;

// Wrapper Objects
Integer c = 7;
Double d = 10.2;
```

Integer and Double Static Fields

The `Integer` and `Double` classes have important static fields named `Integer.MAX_VALUE`, `Integer.MIN_VALUE`, `Double.POSITIVE_INFINITY`, and `Double.NEGATIVE_INFINITY`. These can be used to find the largest and smallest `Integer` values that Java can store as well as represent the concept of infinity.

```
Integer wrapperIntMax = Integer.MAX_VALUE;
Integer wrapperIntMin = Integer.MIN_VALUE;
Double inf = Double.POSITIVE_INFINITY;
Double negInf = Double.NEGATIVE_INFINITY;
```

Static Methods

Static methods are methods that can be called within a program without creating an object of the class.

```
// static method
public static int getTotal(int a, int b) {
    return a + b;
}

public static void main(String[] args) {
    int x = 3;
    int y = 2;
    System.out.println(getTotal(x,y)); //
Prints: 5
}
```

Calling a Static Method

Static methods can be called by appending the dot operator to a class name followed by the name of the method.

```
int largerNumber = Math.max(3, 10); // Call
static method
System.out.println(largerNumber); // Prints:
10
```

The Math Class

The `Math` class (which is part of the `java.lang` package) contains a variety of static methods that can be used to perform numerical calculations.

```
System.out.println(Math.abs(-7.0)); //
Prints: 7
```

```
System.out.println(Math.pow(5, 3)); //
Prints: 125.0
```

```
System.out.println(Math.sqrt(52)); // Prints:
7.211102550927978
```